

گزارش کار فاز دوم پروژه درس طراحی کامپایلر (طراحی غیر مبهم ساز)

(اعضای گروه: امیرعلی محمدپور، ارمیا میرزایی، علی نوروزی)

بخش ۱) شرح کامل تکنیک های پیاده سازی شده

۱) تحلیل و پیاده سازی ماژول حذف کد مرده (Dead Code Elimination)

در این پروژه برای حذف خودکار کدهای مرده (Dead Code) از برنامه های نوشته شده به زبان Mini-C، کلاس DeadCodeAnalyzer بر اساس الگوی Visitor ارائه شده توسط ANTLR طراحی شده است. این کلاس از MiniCVisitor ارث بری کرده و با پیمایش درخت نحوی (AST) تولید شده توسط MiniCParser، بخش هایی از کد که استفاده ای در اجرای نهایی ندارند یا اجرای آنها غیرممکن است را شناسایی و برای حذف علامت گذاری می کند. حذف این بخش ها در انتها با کمک ابزار TokenStreamRewriter انجام شده و نسخه تمیز شده کد در فایل خروجی ذخیره می شود.

فرآیند کار به این صورت است که ابتدا تابع delete_dead_code با استفاده از FileStream، فایل ورودی را خوانده و پس از لکسیکال آنالیز توسط MiniCLexer، یک CommonTokenStream ایجاد می کند. سپس MiniCParser روی این توکن ها اجرا شده و درخت نحوی کامل برنامه (گره (program) تولید می شود. شیء DeadCodeAnalyzer روی این درخت اعمال شده و در طی بازدید از گره های مختلف، اطلاعات لازم برای شناسایی کدهای مرده گردآوری می شود. پس از اتمام پیمایش، بخش های شناسایی شده به ترتیب معکوس موقعیت در متن (برای جلوگیری از تداخل در حذف ها) از طریق TokenStreamRewriter حذف شده و متن بازنویسی شده به عنوان خروجی در فایل مشخص شده ذخیره می شود.

گزارش کار فاز دوم پروژه درس طراحی کامپایلر (طراحی غیر مبهم ساز)

(اعضای گروه: امیرعلی محمدپور، ارمیا میرزایی، علی نوروزی)

در کلاس `DeadCodeAnalyzer`، برای مدیریت متغیرها از ساختاری به نام `scope_vars` استفاده شده که لیستی از دیکشنری‌ها برای نگهداری متغیرهای هر `Scope` است. هنگام ورود به یک بلوک جدید (گره `CompoundStmt`)، یک `Scope` جدید به این لیست افزوده شده و پس از اتمام پردازش بلوک، متغیرهایی که علامت استفاده (`used`) برایشان `False` باقی مانده، به عنوان متغیرهای استفاده نشده شناسایی و محدوده توکن مربوطه در لیست `to_delete` ذخیره می‌شود. این محدوده با استفاده از شاخص‌های `start_token_index` و `stop_token_index` تعیین می‌گردد که به ترتیب موقعیت اولین و آخرین توکن مربوط به اعلان متغیر را نشان می‌دهند.

متغیرهای استفاده شده در طول پیمایش از طریق متد `visitPrimaryExpr` شناسایی می‌شوند. در این متد هرگاه یک `IDENTIFIER` (نام متغیر) در برنامه مشاهده شود، لیست `Scope`‌ها از انتها به ابتدا بررسی شده و در صورت یافتن نام متغیر، وضعیت `used` برای آن برابر `True` قرار می‌گیرد. این رویکرد، سازگاری با مفهوم بلوک‌های تو در تو و محدوده‌های مختلف متغیرها را تضمین می‌کند.

به جز شناسایی متغیرهای بلااستفاده، این ماژول ساختارهای کنترلی با شرط‌های ثابت را نیز هدف قرار می‌دهد. متد `visitSelectionStmt` برای گره‌های شرطی (`if`) و متد `visitIterationStmt` برای حلقه‌های `while` به گونه‌ای پیاده‌سازی شده‌اند که در صورت شناسایی شرطی با مقدار ثابت `false` یا `0`، کل گره مربوطه در لیست حذف قرار گیرد. در چنین شرایطی اجرای بدنه این دستورات هیچ‌گاه اتفاق نخواهد افتاد و حذف آن‌ها ایمنی کد را نقض نمی‌کند.

پس از تحلیل کامل کد، تابع `delete_dead_cod` با مرتب‌سازی محدوده‌های حذف بر اساس شاخص شروع آن‌ها و استفاده از `rewriter.delete(start, stop)`، این بخش‌ها را از توکن‌استریم حذف کرده و نسخه بهینه‌شده برنامه را در فایل خروجی

گزارش کار فاز دوم پروژه درس طراحی کامپایلر (طراحی غیر مبهم ساز)

(اعضای گروه: امیرعلی محمدپور، ارمیا میرزایی، علی نوروزی)

ذخیره می‌کند. این روش باعث می‌شود فاصله‌ها و قالب‌بندی کد اصلی تا حد زیادی حفظ شود و حذف‌ها بدون آسیب به ساختار کلی متن صورت گیرد.

این پیاده‌سازی از نظر پیچیدگی زمانی به صورت خطی نسبت به اندازه درخت نحوی ($O(N)$) عمل می‌کند و به دلیل استفاده از `TokenStreamRewriter` برای ویرایش، سرعت بالایی دارد. با وجود این، برخی محدودیت‌ها در نسخه فعلی وجود دارد:

- در اعلان‌های چندگانه (مثلاً `int a=1, b=2;`) حذف یک متغیر بلااستفاده ممکن است باعث حذف کل خط شود.
- تحلیل شرط‌ها بسیار ساده است و تنها رشته‌های متنی 0 یا false را بررسی می‌کند.
- ساختارهای `if-else` به صورت کامل حذف می‌شوند و در حالت وجود `else`، این بخش نیز حذف خواهد شد.
- امکان وجود اثرات جانبی در `initializer` ها یا شروط وجود دارد که حذف آن‌ها ممکن است رفتار برنامه را تغییر دهد.

با وجود این محدودیت‌ها، این بخش از پروژه به عنوان یک نمونه ساده اما کاربردی از تکنیک **Dead Code Elimination** نشان می‌دهد که چگونه می‌توان با استفاده از قابلیت‌های **ANTLR** و ابزارهایی مانند `TokenStreamRewriter`، برنامه را تحلیل کرده و کدهای غیرضروری را حذف نمود. توسعه‌های آینده شامل اضافه کردن **constant folding** برای تحلیل دقیق‌تر شرط‌ها، بهبود مدیریت اعلان‌های چندگانه و طراحی تحلیل‌های داده جریان برای تشخیص کد مرده در سطوح پیچیده‌تر است.

گزارش کار فاز دوم پروژه درس طراحی کامپایلر (طراحی غیر مبهم ساز)

(اعضای گروه: امیرعلی محمدپور، ارمیا میرزایی، علی نوروزی)

۲) تحلیل و پیاده سازی ماژول ساده سازی عبارات معادل (Equivalent Expression Deobfuscation)

این ماژول با هدف حذف پیچیدگی های ظاهری و ساده سازی عبارات منطقی و ریاضی در کدهای زبان Mini-C طراحی شده است. کلاس `EquivalentExpressionDeobfuscator` از `MiniCVisitor` ارث بری کرده و با پیمایش درخت نحوی برنامه، عبارات ریاضی و منطقی را شناسایی کرده و به صورت معادل اما ساده تر بازنویسی می کند. این کار با استفاده از قابلیت `TokenStreamRewriter` انجام می شود تا بدون تغییر ساختار کلی متن، فقط بخش های موردنظر بازنویسی شوند.

در این پیاده سازی، دو الگوی اصلی ساده سازی پوشش داده شده اند:

۱. ساده سازی جمع و تفریق با اعداد منفی:

اگر یک عبارت به شکل $x - (-y)$ یا $x + (-y)$ باشد، بازنویسی به

ترتیب به $x + y$ و $x - y$ انجام می شود.

۲. ساده سازی منطق! روی عملگرهای مقایسه:

عبارات به فرم $(a == b)$ یا $(a != b)$ به ترتیب به $a == b$ و $a != b$

$b ==$ بازنویسی می شوند.

ساختار کلاس و اجزای کلیدی

۱. مدیریت بازنویسی توکن ها:

کلاس `EquivalentExpressionDeobfuscator` در سازنده خود

`(__init__)` یک نمونه `TokenStreamRewriter` می سازد و از آن

برای جایگزینی بخش های انتخاب شده از متن استفاده می کند. در این روش،

گزارش کار فاز دوم پروژه درس طراحی کامپایلر (طراحی غیر مبهم ساز)

(اعضای گروه: امیرعلی محمدپور، ارمیا میرزایی، علی نوروزی)

شاخص شروع (`start.tokenIndex`) و پایان (`stop.tokenIndex`) هر عبارت تعیین می شود و بازنویسی فقط در همان محدوده انجام می گیرد.

۲. توابع کمکی برای تحلیل متن عبارات:

- متد `strip_outer_parens`: پرانتزهای اضافی در ابتدای و انتهای یک عبارت را حذف می کند، به شرطی که این پرانتزها تمام عبارت را شامل شوند. این روش برای جلوگیری از تغییر غیر ضروری اولویت عملگرها طراحی شده است.
- متد `split_top_level`: عبارت منطقی ورودی را با در نظر گرفتن عمق پرانتزها تحلیل کرده و اگر در سطح بالا (`top-level`) شامل عملگر `==` یا `!=` باشد، سه بخش عبارت (سمت چپ، عملگر، سمت راست) را استخراج می کند.

۳. ساده سازی عبارات ریاضی (متد `visitAdditiveExpr`):
- این متد روی گره های `AdditiveExpr` اعمال می شود. در صورتی که عبارت به شکل $(-expr2) \pm expr1$ باشد:
- اگر عملگر `-` باشد، عبارت به `expr1 + expr2` تغییر می یابد.
 - اگر عملگر `+` باشد، عبارت به `expr1 - expr2` بازنویسی می شود.
- این کار با استفاده از `strip_outer_parens` برای حذف پرانتزهای غیر ضروری انجام می شود.

۴. ساده سازی عبارات منطقی (متد `visitUnaryExpr`):
- در این متد، اگر گره `UnaryExpr` یک `!` داشته باشد و عبارت داخل آن شامل یک مقایسه `==` یا `!=` باشد، عملگر معکوس شده و پرانتزهای اضافی حذف می شوند. به عنوان نمونه:

گزارش کار فاز دوم پروژه درس طراحی کامپایلر (طراحی غیر مبهم ساز)

(اعضای گروه: امیرعلی محمدپور، ارمیا میرزایی، علی نوروزی)

$x \neq y$ به $(x == y)$ ◦

$x == y$ به $(x \neq y)$ ◦

۵. تابع اصلی `expression_simplifier`

این تابع با دریافت مسیر ورودی و خروجی، فایل سورس را پردازش کرده، درخت نحوی را تولید می کند و یک نمونه از `EquivalentExpressionDeobfuscator` روی آن اجرا می کند. سپس متن بازنویسی شده از طریق `TokenStreamRewriter` استخراج و در فایل خروجی ذخیره می شود.

ویژگی ها و مزایا

- حفظ ساختار اصلی متن: با استفاده از `TokenStreamRewriter` تنها بخش های مشخصی از متن بازنویسی می شوند و قالب بندی کلی دست نخورده باقی می ماند.
- الگوریتم تحلیل سطح پرانتزها: توابع کمکی امکان تشخیص صحیح پرانتزهای اضافی و موقعیت دقیق عملگرهای منطقی را فراهم می کنند.
- تمرکز بر ساده سازی بدون تغییر منطق برنامه: تمام بازنویسی ها معادل منطقی یا ریاضی عبارات اولیه هستند، در نتیجه رفتار برنامه تغییر نمی کند.

۳) تحلیل و پیاده سازی ماژول تغییر نام `SemanticRenameVisitor`

کلاس `SemanticRenameVisitor` یکی از اجزای اصلی پروژه `Obfuscation/Deobfuscation` برای زبان `Mini-C` است که با استفاده از `ANTLR4` طراحی شده است. هدف اصلی این کلاس، تغییر نام متغیرها و توابع در کد

گزارش کار فاز دوم پروژه درس طراحی کامپایلر (طراحی غیر مبهم ساز)

(اعضای گروه: امیرعلی محمدپور، ارمیا میرزایی، علی نوروزی)

ورودی به گونه‌ای است که هم ساختار معنایی (Semantic) برنامه حفظ شود و هم نام‌ها به شکلی سیستمی، خوانایی اولیه خود را از دست بدهند. این تکنیک به عنوان **Semantic-Aware Renaming** شناخته می‌شود و در ابزارهای Obfuscator برای دشوارسازی خواندن کد توسط اشخاص ثالث کاربرد دارد.

ساختار کلی کلاس

این کلاس از **MiniCVisitor** ارث‌بری کرده و به کمک **TokenStreamRewriter**، تغییرات لازم را روی **Token Stream** اعمال می‌کند. در این فرآیند، تمامی شناسه‌ها (**Identifiers**) در کد تحلیل شده و نام‌های جدیدی بر اساس **Context** آن‌ها تولید می‌شود. این تغییر نام به صورت **Context-sensitive** است، یعنی شرایط استفاده از متغیر یا تابع، بر انتخاب نام جدید تأثیر دارد. کلاس دارای ویژگی‌های کلیدی زیر است:

- **Rewriter**: برای جایگزینی توکن‌ها با مقادیر جدید استفاده می‌شود.
- **token_stream**: جریان توکن‌های تحلیل شده توسط **Lexer** و **Parser** را نگه می‌دارد.
- **name_map**: یک دیکشنری که نام‌های اصلی را به نام‌های جدید نگاشت می‌دهد.
- **func_count / var_count**: شمارنده‌های داخلی برای تولید نام‌های جدید سیستماتیک.
- **skip_names**: مجموعه‌ای از نام‌ها که نباید تغییر کنند؛ برای مثال، تابع **main** دست‌نخورده باقی می‌ماند.

گزارش کار فاز دوم پروژه درس طراحی کامپایلر (طراحی غیر مبهم ساز)

(اعضای گروه: امیرعلی محمدپور، ارمیا میرزایی، علی نوروزی)

منطق تولید نامها

دو متد مهم برای تولید نامهای جدید در نظر گرفته شده اند:

۱. `_generate_func_name(original)`

این متد نام توابع را تولید می کند. اگر نام تابع اصلی شامل عبارت "print" باشد، به طور پیش فرض نام `printHelper` برای آن انتخاب می شود تا هدف تابع مشخص بماند. در غیر این صورت، توابع به صورت `func1, func2, ...` نام گذاری می شوند.

۲. `_generate_var_name(ctx, original)`

این متد مسئول انتخاب نام برای متغیرها است و با استفاده از **Heuristic Rules** تصمیم گیری می کند:

- اگر متغیر در یک حلقه `ForStmt` یا `WhileStmt` تعریف شده باشد، به عنوان شمارنده حلقه نام هایی مانند `i` یا `j` برای آن انتخاب می شود.
- اگر در یک عبارت انتساب شامل عملگرهای ریاضی باشد، متناسب با عملیات از نام های `sum, diff, product` استفاده می شود.
- برای سایر متغیرها، ابتدا از نام های ساده ای مانند `x, y, z, a, c` و در ادامه از نام های ترتیبی مانند `var0, var1, ...` استفاده می شود.

متدهای Visitor و عملکرد آنها

این کلاس با استفاده از روش **Visitor Pattern** روی **Parse Tree** حرکت کرده و تغییرات لازم را اعمال می کند:

گزارش کار فاز دوم پروژه درس طراحی کامپایلر (طراحی غیر مبهم ساز)

(اعضای گروه: امیرعلی محمدپور، ارمیا میرزایی، علی نوروزی)

- `visitFunctionDecl`: برای تغییر نام توابع به کار می رود. اگر نام تابع در لیست `skip_names` نباشد، با نام جدید جایگزین می شود.
- `visitParam`: تغییر نام پارامترهای ورودی توابع.
- `visitInitDeclarator`: تغییر نام متغیرهایی که در مرحله ی تعریف اولیه مقداردهی می شوند.
- `visitPrimaryExpr`: جایگزینی متغیرها در تمام `Expression` ها بر اساس نگاشت ذخیره شده در `name_map`.
- `visitAssignmentExpr`: جایگزینی نام متغیرهای سمت چپ در عبارات انتساب.

تابع *renamer*

این تابع نقطه ورود برای اجرای این `Visitor` روی یک فایل `Mini-C` است:

۱. فایل ورودی را خوانده و توسط `MiniCLexer` و `MiniCParser` تجزیه می کند.
۲. `SemanticRenameVisitor` روی درخت نحوی اجرا شده و نامها تغییر می کنند.
۳. خروجی تغییر یافته در یک فایل جدید نوشته می شود.

بخش ۲) توجیه تصمیمات طراحی (۵ تا به انتخاب)

۱. استفاده از `ANTLR4` برای `Parsing` زبان `Mini-C`

گزارش کار فاز دوم پروژه درس طراحی کامپایلر (طراحی غیر مبهم ساز)

(اعضای گروه: امیرعلی محمدپور، ارمیا میرزایی، علی نوروزی)

یکی از مهم ترین تصمیم های طراحی در این پروژه، استفاده از ANTLR4 برای ساخت Parser، Lexer و تولید Parse Tree است. این انتخاب باعث شده که منبع زبان Mini-C به صورت ساختاریافته تجزیه شود، نه با Regex یا String Processing ساده. این روش امکان می دهد که تمام تغییرات (مثل تغییر نام متغیرها یا ساده سازی عبارات) به صورت Syntax-aware انجام شوند. با این رویکرد، تمام ساختار نحوی زبان Mini-C به طور دقیق شناسایی می شود و احتمال خراب شدن کد در اثر تغییر نام یا حذف کدهای اضافی به حداقل می رسد.

ANTLR همچنین کد Visitor برای درخت نحوی تولید می کند که کار توسعه دهنده را بسیار ساده کرده است. به جای نوشتن Parser از صفر، ما از Visitor Pattern استفاده می کنیم تا روی هر Node از درخت حرکت کرده و تغییرات اعمال شود. این کار علاوه بر ساده سازی توسعه، باعث می شود که پروژه به راحتی در آینده قابل توسعه به زبان های جدیدتر یا نسخه های متفاوت Mini-C باشد.

در مجموع، انتخاب ANTLR4 باعث شده این پروژه از یک ابزار ساده به یک De-Obfuscator حرفه ای ارتقاء پیدا کند که تغییرات را با درک کامل از Context و Syntax زبان اعمال می کند.

۲. استفاده از TokenStreamRewriter برای بازنویسی امن کد

یک تصمیم کلیدی دیگر در طراحی، استفاده از TokenStreamRewriter برای اعمال تغییرات روی توکن ها است. به جای تغییر مستقیم متن کد، این کلاس توکن های تولید شده توسط Lexer را مدیریت کرده و تغییرات را در سطح Token Stream اعمال می کند. این کار چند مزیت دارد:

اول اینکه از بروز خطاهای نحوی جلوگیری می کند، زیرا تغییرات روی ساختار رسمی

گزارش کار فاز دوم پروژه درس طراحی کامپایلر (طراحی غیر مبهم ساز)

(اعضای گروه: امیرعلی محمدپور، ارمیا میرزایی، علی نوروزی)

زبان اعمال می شوند. دوم، اعمال تغییرات مثل **Rename** یا حذف کد مرده بسیار راحت تر است، چون فقط نیاز به جایگزینی یا حذف توکن ها داریم، نه بازنویسی دستی کل خطوط. سوم، این ابزار امکان بازسازی متن اصلی کد را پس از تغییرات بدون دست کاری دستی فرمت و فاصله ها فراهم می کند، که برای **De-Obfuscation** اهمیت بالایی دارد.

این انتخاب همچنین با **ANTLR Visitor Pattern** کاملاً هماهنگ است، زیرا هر گره از درخت نحوی شامل اطلاعات مربوط به موقعیت توکن هاست. توسعه دهنده می تواند بدون نگرانی از موقعیت دقیق کلمات یا خطوط، تنها با استفاده از ایندکس توکن ها تغییرات را انجام دهد.

در واقع **TokenStreamRewriter** باعث شده پروژه هم ساده تر و هم حرفه ای تر شود و خطر از دست رفتن ساختار کد را به صفر برساند.

۳. طراحی ماژولار و تفکیک وظایف (**Renamer**، **ExpressionSimplifier**، **DeadCodeEliminator**)

یکی از نقاط قوت طراحی این پروژه، تقسیم هر قابلیت اصلی به یک کلاس و فایل جداگانه است. سه قابلیت کلیدی یعنی تغییر نام معنایی (**Renamer**)، ساده سازی عبارات منطقی و ریاضی (**ExpressionSimplifier**)، و حذف کد مرده (**DeadCodeEliminator**) به طور کامل از هم جدا هستند.

این تفکیک باعث می شود که توسعه دهندگان بتوانند هر بخش را به طور مستقل تست و نگهداری کنند. برای مثال، اگر در آینده نیاز به تغییر الگوریتم حذف کد مرده باشد، فقط همان فایل تغییر می کند بدون اینکه روی سایر قسمت ها اثر بگذارد. همچنین، با این طراحی، کاربر در **GUI** می تواند به صورت انتخابی قابلیت های مختلف را

گزارش کار فاز دوم پروژه درس طراحی کامپایلر (طراحی غیر مبهم ساز)

(اعضای گروه: امیرعلی محمدپور، ارمیا میرزایی، علی نوروزی)

فعال یا غیرفعال کند. این انعطاف پذیری در پروژه‌های De-Obfuscation اهمیت زیادی دارد، چون ممکن است در برخی سناریوها فقط به یکی از تکنیک‌ها نیاز باشد. این طراحی Loose Coupling و High Cohesion را در کد برقرار کرده است. در نتیجه، پروژه قابلیت گسترش دارد؛ اضافه کردن تکنیک‌های جدید مثل Control Flow Simplification یا Constant Propagation به راحتی امکان پذیر خواهد بود.

۴. استفاده از Heuristic های معنایی برای انتخاب نام‌ها در Renamer

در بسیاری از ابزارهای Rename، نام متغیرها و توابع به صورت تصادفی یا ترتیبی (مثل var1، var2) انتخاب می‌شوند که خوانایی و منطق کد را از بین می‌برد. اما در این پروژه، برای انتخاب نام‌ها از Heuristic های معنایی استفاده شده است. برای مثال:

- اگر متغیر در یک حلقه باشد، نام‌هایی مانند i یا j انتخاب می‌شود تا معنی شمارنده حلقه حفظ شود.
 - اگر متغیر در عملیات جمع، تفریق یا ضرب استفاده شده باشد، نام‌هایی مثل sum, diff, product انتخاب می‌شوند.
 - برای توابعی که در نامشان print باشد، نام printHelper داده می‌شود.
- این تصمیم باعث می‌شود خروجی De-Obfuscator قابل فهم تر باشد و از تبدیل کد به مجموعه‌ای از نام‌های بی معنی جلوگیری شود. چنین طراحی در پروژه‌های واقعی اهمیت دارد، چون هدف De-Obfuscation، افزایش درک انسانی از کد

گزارش کار فاز دوم پروژه درس طراحی کامپایلر (طراحی غیر مبهم ساز)

(اعضای گروه: امیرعلی محمدپور، ارمیا میرزایی، علی نوروزی)

است، نه فقط تغییر نام. این تصمیم یک تعادل هوشمندانه بین خودکارسازی و حفظ معنای کد ایجاد کرده است.

۵. حذف خطوط خالی برای افزایش خوانایی خروجی

یکی از تصمیم‌های ظاهراً ساده ولی مهم در این پروژه، پیاده‌سازی تابع `remove_all_empty_lines_from_file` برای حذف خطوط خالی پس از انجام De-Obfuscation است. در فرآیندهایی مانند حذف کد مرده یا تغییر نام متغیرها، ممکن است خطوط خالی اضافی در کد باقی بماند که خوانایی خروجی را کاهش می‌دهد. با این تصمیم، خروجی پروژه ساختاریافته‌تر و حرفه‌ای‌تر به نظر می‌رسد. این تابع خطوط ورودی را خوانده، خطوطی که تنها شامل فاصله یا کاراکترهای خالی هستند را حذف کرده و نتیجه را در همان فایل خروجی ذخیره می‌کند. این کار علاوه بر بهبود تجربه کاربر، در فرآیندهای بعدی مانند تحلیل استاتیک کد یا استفاده از ابزارهای دیگر نیز مؤثر است، زیرا ساختار فایل تمیزتر خواهد بود. همچنین این تصمیم نشان‌دهنده توجه توسعه‌دهنده به جزئیات ظاهری و کیفیت خروجی است که معمولاً در پروژه‌های دانشگاهی یا پژوهشی اهمیت زیادی دارد. به طور کلی، این انتخاب ساده، خروجی را برای توسعه‌دهندگان دیگر جذاب‌تر و قابل فهم‌تر کرده است.

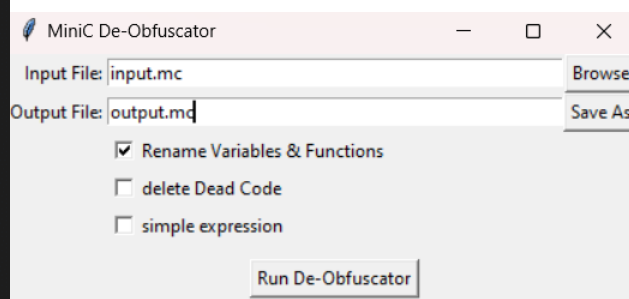
بخش ۳) قبل و بعد چند مثال واقعی

گزارش کار فاز دوم پروژه درس طراحی کامپایلر (طراحی غیر مبهم ساز)

(اعضای گروه: امیرعلی محمدپور، ارمیا میرزایی، علی نوروزی)

ورودی ۱:

```
1 int jIDVlf5() {
2     int OUTBcy = 93;
3     int Pj7a_ = 2;
4     if (!(OUTBcy%2 != 0)) {
5         int Pj7a_ = 1;
6         for (int i = 0; i < 3; i++) {
7             Pj7a_*=i - (-1);
8         }
9         OUTBcy += Pj7a_;
10    } else {
11        int smKUyxx = 0;
12        while (smKUyxx < 5) {
13            OUTBcy += smKUyxx;
14            smKUyxx++;
15        }
16    }
17    int DERoX3PR = OUTBcy - (-Pj7a_);
18    return DERoX3PR*4;
19 }
20
```



خروجی ۱:

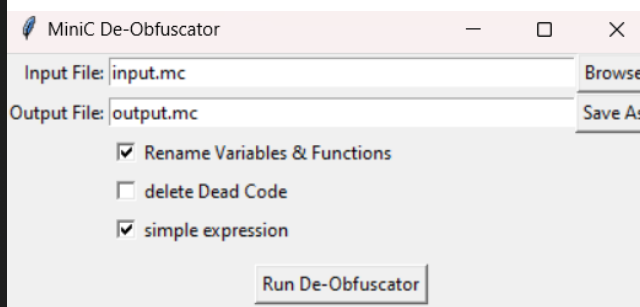
```
1 int func1() {
2     int x = 93;
3     int y = 2;
4     if (!(x%2 != 0)) {
5         int y = 1;
6         for (int i = 0; i < 3; i++) {
7             y*=i - (-1);
8         }
9         x += y;
10    } else {
11        int z = 0;
12        while (z < 5) {
13            x += z;
14            z++;
15        }
16    }
17    int a = x - (-y);
18    return a*4;
19 }
20
```

گزارش کار فاز دوم پروژه درس طراحی کامپایلر (طراحی غیر مبهم ساز)

(اعضای گروه: امیرعلی محمدپور، ارمیا میرزایی، علی نوروزی)

ورودی ۲:

```
1 int jIDVLF5() {
2     int OUTBcy = 93;
3     int Pj7a_ = 2;
4     if (!(OUTBcy%2 != 0)) {
5         int Pj7a_ = 1;
6         for (int i = 0; i < 3; i++) {
7             Pj7a_*=i - (-1);
8         }
9         OUTBcy += Pj7a_;
10    } else {
11        int smKUyxx = 0;
12        while (smKUyxx < 5) {
13            OUTBcy += smKUyxx;
14            smKUyxx++;
15        }
16    }
17    int DERoX3PR = OUTBcy - (-Pj7a_);
18    return DERoX3PR*4;
19 }
20
```



خروجی ۲:

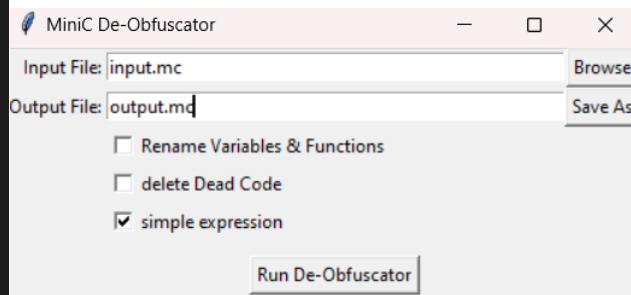
```
1 int func1() {
2     int x = 93;
3     int y = 2;
4     if (x%2 == 0) {
5         int y = 1;
6         for (int i = 0; i < 3; i++) {
7             y*=i + 1;
8         }
9         x += y;
10    } else {
11        int z = 0;
12        while (z < 5) {
13            x += z;
14            z++;
15        }
16    }
17    int a = x + y;
18    return a*4;
19 }
20
```

گزارش کار فاز دوم پروژه درس طراحی کامپایلر (طراحی غیر مبهم ساز)

(اعضای گروه: امیرعلی محمدپور، ارمیا میرزایی، علی نوروزی)

ورودی ۳:

```
1 int jIDVlf5() {
2     int OUTBcy = 93;
3     int Pj7a_ = 2;
4     if (!(OUTBcy%2 != 0)) {
5         int Pj7a_ = 1;
6         for (int i = 0; i < 3; i++) {
7             Pj7a_*=i - (-1);
8         }
9         OUTBcy += Pj7a_;
10    } else {
11        int smKUyxx = 0;
12        while (smKUyxx < 5) {
13            OUTBcy += smKUyxx;
14            smKUyxx++;
15        }
16    }
17    int DERoX3PR = OUTBcy - (-Pj7a_);
18    return DERoX3PR*4;
19 }
20
```



خروجی ۳:

```
1 int jIDVlf5() {
2     int OUTBcy = 93;
3     int Pj7a_ = 2;
4     if (OUTBcy%2 == 0) {
5         int Pj7a_ = 1;
6         for (int i = 0; i < 3; i++) {
7             Pj7a_*=i + 1;
8         }
9         OUTBcy += Pj7a_;
10    } else {
11        int smKUyxx = 0;
12        while (smKUyxx < 5) {
13            OUTBcy += smKUyxx;
14            smKUyxx++;
15        }
16    }
17    int DERoX3PR = OUTBcy + Pj7a_;
18    return DERoX3PR*4;
19 }
20
```

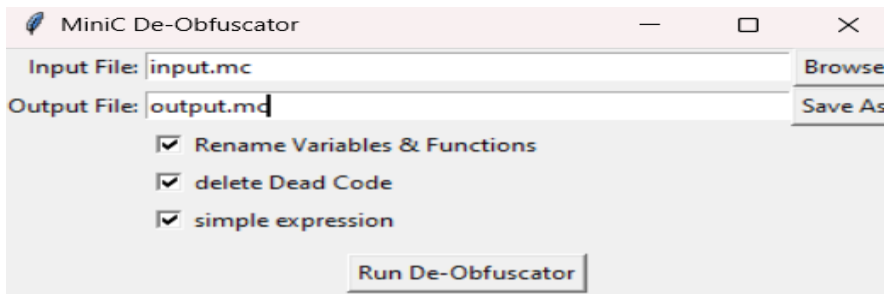
گزارش کار فاز دوم پروژه درس طراحی کامپایلر (طراحی غیر مبهم ساز)

(اعضای گروه: امیرعلی محمدپور، ارمیا میرزایی، علی نوروزی)

ورودی ۴:

```
1 int tLB(int OUTBcy) {
2     return OUTBcy * 100;
3     while (0) {
4         int KKA = 94;
5     }
6     bool k1Culo5R = true;
7     bool kPb = false;
8 }
9 int jIDVlf5() {
10    int OUTBcy = 93;
11    int Pj7a_ = 2;
12    if (!OUTBcy%2 != 0) {
13        int Pj7a_ = 1;
14        for (int i = 0; i < 3; i++) {
15            Pj7a_*=i - (-1);
16        }
17        OUTBcy += Pj7a_;
18    } else {
19        int smKUyxx = 0;
20        while (smKUyxx < 5) {
21            OUTBcy += smKUyxx;
22            smKUyxx++;
23        }
24    }
25    int DERoX3PR = OUTBcy - (-Pj7a_);
26    return DERoX3PR*4;
27 }
28 int main() {
29     int selector_2050 = 1;
30     while (selector_2050 > 0) {
31         switch(selector_2050) {
32             case 1:
33                 int Wjz3IEN = 3;
34                 selector_2050 = 2;
35                 break;
36             case 2:
37                 int bpf = 5;
38                 selector_2050 = 3;
39                 break;
40             case 3:
41                 int jKXJq = Wjz3IEN - (-bpf);
42                 selector_2050 = 4;
43                 break;
44             case 4:
45                 int DERoX3PR = bpf + (-Wjz3IEN);
46                 selector_2050 = 5;
47                 break;
48             case 5:
49                 int Ghp = first(bpf);
50                 selector_2050 = 6;
51                 break;
52             case 6:
53                 printf("%d\n", Wjz3IEN - (-bpf));
54                 selector_2050 = 7;
55                 break;
56             case 7:
57                 if (0) {
58                     jKXJq = 46;
59                     Ghp = 53;
60                 }
61                 selector_2050 = 8;
62                 break;
63             case 8:
64                 char dm_2M = 'i';
65                 selector_2050 = 9;
66                 break;
67             case 9:
68                 return 0;
69                 selector_2050 = 10;
70                 break;
71             case 10:
72                 if (0) {
73                     bpf = 60;
74                 }
75                 selector_2050 = 11;
76                 break;
77             case 11:
78                 int mmDzuGh = 56;
79                 selector_2050 = 12;
80                 break;
81             case 12:
82                 selector_2050 = 0;
83                 break;
84         }
85     }
86 }
87 int qDhnrnT() {
88     int OUTBcy = 86;
89     int Pj7a_ = 9;
90     if (!OUTBcy%2 != 0) {
91         int Pj7a_ = 1;
92         for (int i = 0; i < 3; i++) {
93             Pj7a_*=i - (-1);
94         }
95         OUTBcy += Pj7a_;
96     } else {
97         int smKUyxx = 0;
98         while (smKUyxx < 5) {
99             OUTBcy += smKUyxx;
100             smKUyxx++;
101         }
102     }
103     int DERoX3PR = OUTBcy - (-Pj7a_);
104     return DERoX3PR*4;
105 }
```

خروجی ۴:



```
1 int main() {
2     int var7 = 1;
3     while (var7 > 0) {
4         switch(var7) {
5             case 1:
6                 int var8 = 3;
7                 var7 = 2;
8                 break;
9             case 2:
10                int var9 = 5;
11                var7 = 3;
12                break;
13             case 3:
14                var7 = 4;
15                break;
16             case 4:
17                var7 = 5;
18                break;
19             case 5:
20                var7 = 6;
21                break;
22             case 6:
23                printf("%d\n", var8 + var9);
24                var7 = 7;
25                break;
26             case 7:
27                var7 = 8;
28                break;
29             case 8:
30                var7 = 9;
31                break;
32             case 9:
33                return 0;
34                var7 = 10;
35                break;
36             case 10:
37                var7 = 11;
38                break;
39             case 11:
40                var7 = 12;
41                break;
42             case 12:
43                var7 = 0;
44                break;
45         }
46     }
47 }
```

گزارش کار فاز دوم پروژه درس طراحی کامپایلر (طراحی غیر مبهم ساز)

(اعضای گروه: امیرعلی محمدپور، ارمیا میرزایی، علی نوروزی)

بخش ۴) چالش ها و نکات مهم

۱. طراحی ماژولار و مدیریت وابستگی ها

یکی از نکات اساسی این پروژه، طراحی ماژولار برای پیاده سازی تکنیک های مختلف **De-Obfuscation** است. در این پروژه، هر تکنیک در یک فایل جداگانه پیاده سازی شده است:

- **Renamer** برای تغییر نام متغیرها و توابع
- **DeadCodeEliminator** برای حذف کدهای مرده
- **ExpressionSimplifier** برای ساده سازی عبارات

این ساختار باعث می شود که توسعه دهنده بتواند به سادگی یک تکنیک جدید اضافه یا یک تکنیک موجود را بدون تأثیر بر سایر بخش ها تغییر دهد. این امر با اصل **Separation of Concerns** و **Single Responsibility Principle** همخوانی دارد.

اما همین ماژولار بودن یک چالش نیز ایجاد می کند: مدیریت وابستگی ها. برای مثال، در این پروژه، تمام تکنیک ها از طریق `main.py` فراخوانی می شوند، به طوری که هر تکنیک باید قابلیت کار با فایل ورودی و خروجی مشخص را داشته باشد. اگر یکی از ماژول ها به ساختار خاصی از کد وابسته باشد (برای مثال، فرض کند کد قبلاً توسط یک تکنیک دیگر اصلاح شده است)، این وابستگی ها ممکن است باعث بروز **Bug** یا ناهمخوانی شوند. بنابراین، نیاز است که ماژول ها طوری طراحی شوند که:

گزارش کار فاز دوم پروژه درس طراحی کامپایلر (طراحی غیر مبهم ساز)

(اعضای گروه: امیرعلی محمدپور، ارمیا میرزایی، علی نوروزی)

۱. به صورت مستقل کار کنند.
۲. از رابط‌های استاندارد برای ورودی و خروجی استفاده کنند) در اینجا فایل با فرمت Mini-C.
۳. در صورت نیاز به توسعه، فقط در `main.py` تغییر ایجاد شود و ماژول‌ها مستقل از هم باقی بمانند.

این ساختار پروژه از نظر توسعه‌پذیری (Scalability) و نگهداری (Maintainability) بسیار ارزشمند است. هر تکنیک می‌تواند توسط یک توسعه‌دهنده جداگانه مدیریت شود، که این موضوع برای پروژه‌های تیمی اهمیت بالایی دارد. همچنین، وجود ماژول‌های جداگانه باعث شده کد خواناتر و درک آن ساده‌تر شود. در عین حال، یکی از چالش‌ها این است که تست و Debug هر بخش نیازمند پوشش تستی دقیق است تا از همخوانی همه ماژول‌ها در کنار هم اطمینان حاصل شود.

۲. طراحی رابط کاربری (GUI) ساده و کاربرپسند با استفاده از Tkinter

انتخاب Tkinter برای رابط گرافیکی پروژه از مهم‌ترین نکات طراحی است. این کتابخانه به صورت پیش فرض با Python ارائه می‌شود و برای پروژه‌هایی که نیاز به رابط کاربری سبک دارند، انتخابی ایده‌آل است. در این پروژه، رابط کاربری ساده‌ای طراحی شده که به کاربر اجازه می‌دهد:

۱. فایل ورودی را انتخاب کند.
۲. مسیر ذخیره‌سازی فایل خروجی را تعیین کند.
۳. انتخاب کند کدام تکنیک‌ها روی فایل اعمال شوند.
۴. اجرای فرآیند را تنها با یک کلیک (Run De-Obfuscator) انجام دهد.

گزارش کار فاز دوم پروژه درس طراحی کامپایلر (طراحی غیر مبهم ساز)

(اعضای گروه: امیرعلی محمدپور، ارمیا میرزایی، علی نوروزی)

این سادگی رابط کاربری باعث شده حتی کاربرانی که آشنایی کمی با Python یا خط فرمان دارند، بتوانند از ابزار استفاده کنند. طراحی مینیمال این رابط از اصول User-Centered Design پیروی می کند و بدون پیچیدگی اضافه، تمام نیازهای کاربر را برطرف می کند.

اما این انتخاب چالش هایی هم دارد:

- محدودیت در طراحی ظاهری Tkinter: به دلیل سبک بودن، امکانات محدودی در طراحی ظاهر حرفه ای دارد.
- مدیریت خطاها: برای جلوگیری از کرش کردن برنامه در صورت انتخاب مسیر اشتباه یا مشکلات دیگر، باید مکانیزم مدیریت خطا (try-except) در تمام بخش ها رعایت شود.
- پشتیبانی Cross-Platform: هرچند Tkinter روی سیستم های مختلف کار می کند، اما رندر شدن رابط در سیستم عامل های مختلف ممکن است کمی متفاوت باشد.

با وجود این محدودیت ها، استفاده از Tkinter انتخابی هوشمندانه بوده، زیرا باعث کاهش پیچیدگی و افزایش سهولت استفاده شده است. برای پروژه های سبک دانشگاهی و پژوهشی، این انتخاب هم در زمان توسعه صرفه جویی می کند و هم به سادگی انتشار پروژه کمک می کند.

گزارش کار فاز دوم پروژه درس طراحی کامپایلر (طراحی غیر مبهم ساز)

(اعضای گروه: امیرعلی محمدپور، ارمیا میرزایی، علی نوروزی)

۳. چالش در مدیریت جریان داده‌ها و بهینه‌سازی خروجی

در این پروژه، یکی از بخش‌های جالب و مهم، نحوه مدیریت جریان داده‌ها و تولید خروجی نهایی است. برنامه به جای تغییر مستقیم فایل اصلی، ابتدا یک کپی از فایل ورودی می‌سازد و سپس تمام تکنیک‌ها روی فایل کپی شده اعمال می‌شوند. این طراحی چند مزیت مهم دارد:

۱. فایل اصلی دست‌نخورده باقی می‌ماند و امنیت داده‌ها تضمین می‌شود.
۲. خروجی نهایی به صورت زنجیره‌ای از تغییرات حاصل می‌شود و نیازی نیست کاربر هر تکنیک را به صورت دستی اجرا کند.
۳. این ساختار باعث شده برنامه قابلیت توسعه داشته باشد؛ به طوری که اگر تکنیک جدیدی اضافه شود، تنها کافی است آن را در `run_DeObfuscator` فراخوانی کنیم.

در انتها نیز تابع `remove_all_empty_lines_from_file` برای حذف خطوط خالی اضافه شده به کار می‌رود. این موضوع از نظر ظاهری ساده است، اما برای خوانایی کد خروجی بسیار مهم است. در فرآیندهای `De-Obfuscation` و `Obfuscation`، معمولاً تغییرات زیادی در کد ایجاد می‌شود که منجر به ایجاد خطوط خالی، فاصله‌های اضافی یا حتی ساختار نامرتب می‌گردد. وجود این تابع باعث شده خروجی تمیز، خوانا و حرفه‌ای باشد، که برای ادامه کار روی کد (مانند تحلیل استاتیک یا اجرای دوباره) اهمیت زیادی دارد.

اما مدیریت جریان داده‌ها در این پروژه چالش‌های خاصی دارد:

- در صورتی که یکی از تکنیک‌ها خطا دهد، باید مکانیزم مناسبی برای بازگرداندن وضعیت فایل به حالت قبلی طراحی شود.

گزارش کار فاز دوم پروژه درس طراحی کامپایلر (طراحی غیر مبهم ساز)

(اعضای گروه: امیرعلی محمدپور، ارمیا میرزایی، علی نوروزی)

- هماهنگی بین ماژول‌ها اهمیت زیادی دارد؛ زیرا ممکن است حذف کد مرده باعث تغییر شماره خطوط شود یا ساده‌سازی عبارات باعث تغییر ساختار دستورات شود که اجرای تکنیک بعدی را پیچیده‌تر کند.
- استفاده از فایل‌های موقت (Temporary Files) می‌تواند یکی از بهبودهای آینده باشد تا علاوه بر امنیت، خطایابی آسان‌تر شود.

این بخش از پروژه به‌خوبی نشان می‌دهد که طراح کد به بهینه‌سازی تجربه کاربر و قابلیت نگهداری پروژه توجه داشته است.